

NOTE



S

W

e

lec 01 Information representation.

1. 进制之间的转换.

R进制 $\rightarrow$ 10进制 = 展开即可

10进制 $\rightarrow$ R进制

- 整数 = 除基取余, 从下至上
- 小数 = 乘基取整, 从上至下

2进制 $\rightarrow$ 八/十六进制 = 每三/四个一组转换为八/十六进制 (反之同理)

八进制 $\rightarrow$ 十六进制 = 通过二进制

2. 数据的表示.

III 数值数据.

① 定点数. n位机器码的表示范围: 无符号: $0 \leq X \leq 2^n - 1$

(Fixed-point)

有符号 = 取决于编码方法.

▷ 原码 (sign-and-magnitude)

只有符号位改变.

e.g. $3 = 0000\ 0011$

$-3 = 1000\ 0011$

8bit的范围 = -127 (11111111) $\sim$ 127 (01111111)

缺点: 有两种表示0的方式 $\left\{ \begin{array}{l} 1000\ 0000 (-0) \\ 0000\ 0000 (+0) \end{array} \right.$

▷ 反码 (1's complement)

正数
不变

★ 对于基r N的反码: $(r^n - 1) - N$ (n位)

r=2 每一位取反.

8bit的范围: -127 (1000 0000) $\sim$ 127 (0111 1111)

▷ 补码 (2's complement)

★ 对于基r N的补码: $r^n - N$

r=2 取反加1

8bit的范围: -128 (1000 0000) $\sim$ 127 (0111 1111)

▷ 移码 (Frameshift code)

应用: 用于表示浮点数的指数部分.

★ 符号扩展: 扩展时复制最高位.

$0001 \rightarrow 0000\ 0001$

$1001 \rightarrow 1111\ 1001$

$\left\{ \begin{array}{l} \text{扩展 (expanding)} \\ \text{截断 (truncating)} \end{array} \right. \left\{ \begin{array}{l} \text{有符号: 符号扩展 (如上)} \\ \text{无符号: 补0} \end{array} \right. \left. \begin{array}{l} \\ \end{array} \right\} \text{均能得到预期结果}$

截断 (truncating) = 直接去掉. (较小的数能得到预期结果)

② 浮点数 (floating-point)



单精度	1	8	23	Bias=127
双精度	1	11	52	Bias=1023

$$x = (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})} \leftarrow \text{规格化数}$$

★ 转换时，实际指数 + Bias

★ 表示范围 (single)



★ 指数的范围

-126 ~ 127 (0000 0001 ~ 1111 1110)
0000 0000 与 1111 1111 被保留

阶码 Exp	全0	全1	全1	全1
尾数 Frac	全0	非0	全0	非0
表示的数	+0/-0	非规格化	+∞/-∞	NaN

★ 非规格化时的数: $x = (-1)^S \times (0 + \text{Fraction}) \times 2^{1 - \text{Bias}}$

↳ 有两种表示0的方式 ±0.0

★ 舍入 (Rounding)

} 大于一半，向上舍入 e.g. 7.89500 → 7.90
 } 小于一半，向下舍入 7.88500 → 7.88
 } 等于一半，向偶数舍入

③ 十进制的二进制表示 (BCD码)

有权 BCD 码: 8421 码

无权 BCD 码: 格雷码, 余3码

格雷码	0000	0100	0101	0111	0110	0010	0011
十进制	0	1	2	3	4	5	6
	0001	1001	1000				
	7	8	9				

④ 非数值数据

ASCII 码 和 Unicode

汉字的表示: 输入码, 内码, 字型码

3. 数据的长度和存储

① 主存容量单位 1Kb = 2¹⁰ byte 2¹⁰ → MB → GB → TB → PB → EB → ZB → YB

② 频率和线宽单位 b/s → 10³ → kb/s → Mb/s → Gb/s → Tb/s

③ 数据在存储和排列顺序

e.g. 存储 0x01234567

大端序	0x100	0x101	0x102	0x103
	01	23	45	67
小端序	67	45	23	01

lecture 02 数字逻辑

1. 门电路

2. 逻辑函数的化简 ① 卡诺图 ② 真值表

$$\begin{cases} X \oplus Y = XY + \bar{X}\bar{Y} \\ X \odot Y = XY + \bar{X}\bar{Y} \end{cases}$$

e.g. 奇偶校验生成器

★ 成本 (ppt 最后)

① 文字成本: 变量或其补变量出现的次数 L

② 门输入成本 C, CN (加反相器)

三态缓冲器 $EN=0 \Rightarrow Hi-Z$ (高阻态)



lecture 03 组合电路

★ 基本组合电路

① Decoder 译码器

二进制 $\rightarrow$ 十进制

② Encoder 编码器

十进制 $\rightarrow$ 二进制

③ Multiplexer 多路选择器 $\Rightarrow$ e.g. 可进行位扩展

由 Decoder 组成

e.g. 3个开关-1个灯

余3码

加法器

7段数码管

} Decoder 下

★ 1位加法器

★ 实现所有组合电路利用的

① 译码器 + or

② 多路选择器

lecture 04 运算单元

1. 加法

① 1位

半加器

全加器 (有上一位进位)

② 多位

行波进位加法器 (1位一位实现)

超前进位加法器 P13 (多位一起实现)

前缀加法器?

2. 减法

① 无符号 = 被减数 + 减数的补码 (进位为1, right)

为0, false $\Rightarrow$ 再取补码, 并加反号

★ 重要标志位 P42

3. ALU

★ 移位器 P50

① 逻辑移位 $\gg$ (补0)

$A \ll N = A \times 2^N$

$A \gg N = A \div 2^N$

② 算术移位 $\gg$

低位补0

高位补符号位

③ 循环移位 ROR 11001 ROR 2 = 0110

★ 加/减法器

$$a + (b \oplus c) + c$$

$$\begin{cases} C=1 \Rightarrow \text{subtract} \\ C=0 \Rightarrow \text{add} \end{cases}$$

n位数据寄存器
做几次?

Multiplication

① unsigned

分别存储 } product / Multiplicand / ALU = 64位
(n=2) | Multiplier = 32位.

product = product + multiplicand

Multiplier 右移一位.

Multiplicand 左移一位.

Let's do 0010 x 0110 (2 x 6), unsigned

Iteration	Implementation 1			
	Step	Multiplier	Multiplicand	Product
0	initial values	0110	0000 0010	0000 0000
1	1: 0 -> no op	0110	0000 0010	0000 0000
	2: Multiplier shift right/ Multiplicand shift left	011	0000 0100	0000 0000
2	1: 1 -> product = product + multiplicand	011	0000 0100	0000 0100
	2: Multiplier shift right/ Multiplicand shift left	01	0000 1000	0000 0100
3	1: 1 -> product = product + multiplicand	01	0000 1000	0000 1100
	2: Multiplier shift right/ Multiplicand shift left	0	0001 0000	0000 1100
4	1: 0 -> no op	0	0001 0000	0000 1100
	2: Multiplier shift right/ Multiplicand shift left		0010 0000	

与 product 的左半部分相加. } Multiplier / Multiplicand / ALU = 32
(n=2) (分别存储) | product = 64位.

product 的高位 + Multiplicand x Multiplier[0]

product 右移一位

Multiplicand 不变

Multiplier 右移一位.

Let's do 0010 x 0110 (2 x 6), unsigned

Iteration	Implementation 2			
	Step	Multiplier	Multiplicand	Product
0	initial values	0110	0010	0000 _{xxx}
1	1: 0 -> no op	0110	0010	0000 _{xxx}
	2: Multiplier shift right/ Product shift right	_x 011	0010	0000 0 _{xxx}
2	1: 1 -> product = product + multiplicand	_x 011	0010	0010 0 _{xxx}
	2: Multiplier shift right/ Product shift right	_{xx} 01	0010	0001 00 _{xx}
3	1: 1 -> product = product + multiplicand	_{xx} 01	0010	0011 00 _{xx}
	2: Multiplier shift right/ Product shift right	_{xxx} 0	0010	0001 100 _x
4	1: 0 -> no op	_{xxx} 0	0010	0001 100 _x
	2: Multiplier shift right/ Product shift right	_{xxxx}	0010	0000 1100

3) Product 与 Multiplier 合起来存储 } Multiplicand/ALU=32 位.

(n次)

Product + Multiplier = 64 位.

product + Multiplier 的高 32 位 + Multiplicand x Multiplier [0]

product + Multiplier 右移一位.

- Let's do 0010 x 0110 (2 x 6), unsigned

Iteration	Implementation 3			
	Step	Multiplier	Multiplicand	Product/Multiplier
0	initial values	0110	0010	0000 0110
1	1: 0 -> no op	0110	0010	0000 0110
	2: Multiplier shift right/ Product shift right	011	0010	0000 0111
2	1: 1 -> product = product + multiplicand	011	0010	0010 0011
	2: Multiplier shift right/ Product shift right	01	0010	0001 0001
3	1: 1 -> product = product + multiplicand	01	0010	0011 0001
	2: Multiplier shift right/ Product shift right	00	0010	0001 1000
4	1: 0 -> no op	0	0010	0001 1000
	2: Multiplier shift right/ Product shift right	0	0010	0000 1100

② signed = Booth's Algorithm

(n次)

product 与 Multiplier 合起来存储.

看相邻两位 (开始时为 1 则右边补一个 0)

10 product = product - multiplicand (高 32 位)

01 product = product + multiplicand

11/00 nothing

product 右移一位 并且左边补符号位 (最左一位)

Let's do 0010 x 1101 (2 x -3)

Iteration	Implementation 3		
	Step	Multiplicand	Product
0	initial values	0010	0000 1101 0
1	10 -> product = product - multiplicand	0010	1110 1101 0
	shift right		1111 0110 1
2	01 -> product = product + multiplicand	0010	0001 0110 1
	shift right		0000 1011 0
3	10 -> product = product - multiplicand	0010	1110 1011 0
	shift right		1111 0101 1
4	11 -> no op	0010	1111 0101 1
	shift right		1111 1010 1

Division

① unsigned

存储在一个地方

分别存储

Divisor / Dividend / Remainder / ALU = 64 位

!!! (n+1)次

Quotient = 32 位.

Divisor 放在高 32 位.

(恢复)

$\text{Remainder}(\text{Dividend}) - \text{divisor} < 0 \Rightarrow \text{remainder 不变}, Q_0 = 0$

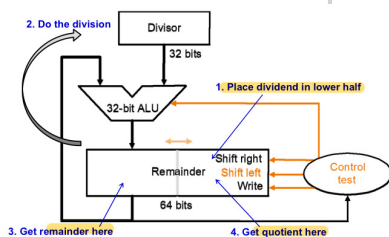
$\text{Remainder}(\text{Dividend}) - \text{divisor} \geq 0 \Rightarrow \text{remainder} = \text{remainder} - \text{divisor}$
 $Q_0 = 1$

Divisor 右移一位

Quotient 左移一位.

Let's do $7 \div 2$

Iteration	Step	Quotient	Divisor	Remainder
0	Initial values	0000	0010 0000	0000 0111
1	1: Rem=Rem-Div	0000	0010 0000	1110 0111
	2b: Rem<0=>+Div, sll Q, Q ₀ =0	0000	0010 0000	0000 0111
	3: Shift Div right	0000	0001 0000	0000 0111
2	1: Rem=Rem-Div	0000	0001 0000	1111 0111
	2b: Rem<0=>+Div, sll Q, Q ₀ =0	0000	0001 0000	0000 0111
	3: Shift Div right	0000	0000 1000	0000 0111
3	1: Rem=Rem-Div	0000	0000 1000	1111 1111
	2b: Rem<0=>+Div, sll Q, Q ₀ =0	0000	0000 1000	0000 0111
	3: Shift Div right	0000	0000 0100	0000 0111
4	1: Rem=Rem-Div	0000	0000 0100	0000 0011
	2a: Rem≥0=> sll Q, Q ₀ =1	0001	0000 0100	0000 0011
	3: Shift Div right	0001	0000 0010	0000 0011
5	1: Rem=Rem-Div	0001	0000 0010	0000 0001
	2a: Rem≥0=> sll Q, Q ₀ =1	0011	0000 0010	0000 0001
	3: Shift Div right	0011	0000 0001	0000 0001



>> improvement } Divisor/ALU = 32 位

(Restoring) remainder + quotient/dividend = 64 位.

所有步骤开始前先将 remainder 左移一位.

$\text{Remainder} - \text{divisor} (\text{高位}) < 0 \Rightarrow \text{Remainder 不变}, \text{恢复后左移一位右边补0}$

$\text{Remainder} - \text{divisor} \geq 0 \Rightarrow \text{Remainder} = \text{Remainder} - \text{divisor}$

左移一位, 右边补1

所有步骤结束后 Remainder 右移一位

$\Rightarrow \text{Remainder 高32位} = \text{余数}; \text{低32位} = \text{商}$

Iteration	Divisor	Divide algorithm	
		Step	Remainder
0	0010	Initial values	0000 0111
	0010	Shift Rem left 1	0000 1110
1	0010	2: Rem = Rem - Div	1110 1110
	0010	3b: $\text{Rem} < 0 \Rightarrow + \text{Div}, \text{sll R}, \text{R0} = 0$	0001 1100
2	0010	2: Rem = Rem - Div	1111 1100
	0010	3b: $\text{Rem} < 0 \Rightarrow + \text{Div}, \text{sll R}, \text{R0} = 0$	0011 1000
3	0010	2: Rem = Rem - Div	0001 1000
	0010	3a: $\text{Rem} \geq 0 \Rightarrow \text{sll R}, \text{R0} = 1$	0011 0001
4	0010	2: Rem = Rem - Div	0001 0001
	0010	3a: $\text{Rem} \geq 0 \Rightarrow \text{sll R}, \text{R0} = 1$	0010 0011
Done	0010	shift left half of Rem right 1	0001 0011

3) non-restoring Division

Iteration	Step	Divisor	Remainder
0	Initial values	0010	0000 0111
	Shift rem left 1	0010	0000 1110
1	2: Rem = Rem - Div	0010	1110 1110
	3b: Rem < 0 (add next), sll 0	0010	1101 1100
2	2: Rem = Rem + Div	0010	1111 1100
	3b: Rem < 0 (add next), sll 0	0010	1111 1000
3	2: Rem = Rem + Div	0010	0001 1000
	3a: Rem > 0 (sub next), sll 1	0010	0011 0001
4	Rem = Rem - Div	0010	0001 0001
	Rem > 0 (sub next), sll 1	0010	0010 0011
	Shift Rem right by 1	0010	0001 0011

减除数。
不满足 (< 0)
左移一位。
加除数。
 > 0
减除数。

2. signed (只能将符号单独拿出来)

☆ (余数与除数相同)